UNDER REVIEW

RESCIENCE C

Replication / Geosciences

# [Re] An open and extensible framework for spatially explicit land use change modelling: the lulcc R package

Sérgio Souza Costa[1, ID]

[1] LambdaGeo, Universidade Federal do Maranhão (UFMA), São Luís, Brazil

Edited by
(Editor)

Received
–

Published
–

DOI
–

## 1 Introduction

Moulds, Buytaert, and Mijic[1] presented lulcc, an R package for spatially explicit land use change modelling, with an explicit aim: to make land use change models open, so that their results can be reproduced and their code examined and extended. They demonstrated the package on the Plum Island Ecosystems data set, which ships with it, going through the stages of an inductive model: suitability from logistic regression, demand from interpolation between observed maps, spatial allocation by CLUE-S[2] and by an ordered procedure[3], and pattern validation by three-map comparison[4].
We replicate that application in Python: a replication in the sense of National Academies of Sciences, Engineering, and Medicine[5], with new code and our own runs, rather than a reproduction with the original code. Both lulcc and the CLUE-S implementation we use were written from the published description of the algorithm[2], independently of each other. Allocation procedures are known to leave room for different readings, with consequences for the simulated maps[6]; the comparison therefore tests two independent readings of the same description, with lulcc's own demonstration as the reference.

## 2 Replication summary

The replication runs on DisSModel[7], a Python framework for spatially explicit dynamic models, and its land use package DisSLUCC[8]. The code is original. Its parts have three origins:

- *CLUE-S allocation*: the component of DisSLUCC 0.5.0, used unchanged. It was ported from LuccME[9], the land use extension of TerraME[10] developed at INPE, and not from lulcc; DisSLUCC is verified against LuccME separately[11].
- *Written for this replication from the original article*: data reading, suitability, demand, the random perturbation of CLUE-S, the ordered allocation, and the three-map comparison with its agreement budget and figure of merit.
- *Conventions taken from lulcc's source code*: where the article leaves a choice open, we read the source and followed it (Section 4). No lulcc code was translated or reused.

Given the same inputs, our CLUE-S and ordered allocations reproduce every simulated map of lulcc cell by cell, and our validation tables agree to $10^{-16}$. With independent

Copyright © 2026 S.S. Costa, released under a Creative Commons Attribution 4.0 International license.
Correspondence should be addressed to Sérgio Souza Costa (sergio.costa@ufma.br)
The authors have declared that no competing interests exist.
Code is available at https://github.com/LambdaGeo/rescience-lulcc..
Data is available at https://github.com/LambdaGeo/lulcc-goldens.

random numbers, 30 runs of each implementation agree in distribution on all 20 metrics compared, under criteria recorded before the runs. Running the original demonstration script required one repair, unrelated to the model (Section 9).

## 3 The original model

The Plum Island Ecosystems data set[12] covers a site in north-east Massachusetts on a grid of $434 \times 497$ cells of about 100 m (99.92 m by 99.95 m), 113 563 of them inside the study area. It holds land use maps for 1985, 1991 and 1999 (forest, built, other), treated as time points 0, 6 and 14, and three explanatory variables: elevation, slope and distance to built land in 1985.

**Suitability.** One binary logistic regression per class is fitted on a stratified random sample of 10% of the cells of the 1985 map: built on the three variables, forest and other on elevation and slope. The paper also fits regression trees and random forests and compares the three model types by the area under the ROC curve (AUC) on the remaining cells (its Fig. 5), and evaluates the glm for built on the cells that became built between 1985 and 1991 (its Fig. 6).

**Demand.** The number of cells of each class at every time point from 0 to 14 is interpolated linearly between the three observed maps and rounded to integers that sum to the number of cells.

**CLUE-S allocation.** At each time step every cell receives, for each class, its suitability plus an elasticity (0.2 for all classes) if the cell already belongs to that class. Each cell is allocated to the class with the highest value plus a class-specific iteration term; the term is then increased by the difference between demand and allocated area, times a scale factor ($10^{-6}$), until the largest difference is at most 50 cells. At every iteration the suitability is perturbed by a uniform random number in $[-0.0002, 0.0002]$ (`jitter.f`).

**Ordered allocation.** Classes are visited in a fixed order (built, forest, other). A class with rising demand takes the most suitable cells not yet settled; a class with falling demand releases its least suitable cells. In the stochastic version a cell qualifies only if its suitability exceeds (or, when releasing, falls below) a uniform random number.

**Validation.** The initial observed map (1985), the final observed map (1999) and the simulated map for 1999 are compared at eight resolutions, aggregating $2 \times 2$ to $256 \times 256$ cells, through the three-dimensional contingency table of Pontius, Peethambaram, and Castella[4]. The paper reports the agreement budget and the figure of merit of the forest-to-built transition for both allocation methods (its Figs. 7 and 8).

## 4 Analysis of the original paper

The article states the model, its parameters and the code that produced its figures, which the package ships as a demonstration script (`demo/gmd-paper.R`). Four aspects are nonetheless not determined by the text.

**Random numbers without a seed.** Three steps draw random numbers: the training sample, the CLUE-S perturbation and the stochastic ordered allocation. The script sets no seed, so the numbers printed in the paper are one draw from a distribution and

cannot be repeated exactly, not even with lulcc. This is why we generated the reference ourselves and compare at two levels (Section 7).

**Choices left to the code.** The text describes the following steps at the level of the algorithm. We followed lulcc's source code for them, so that a difference in the comparison would point at the algorithm rather than at a convention:

- demand rounding: rows rescaled to the number of cells, floored, and the remaining cells given to the classes with the largest remainders;
- training sample: the class codes are numeric, so the stratification of `caret::createDataPartition`, which lulcc calls, groups them by quantile, which for this map gives two strata, {forest, built} and {other};
- ordered allocation: which cells are candidates, the temporary class for released cells, the limit of 1000 random draws, and ties kept in cell order;
- three-map comparison: the layout of the table, aggregation blocks anchored at the upper-left corner with incomplete edge blocks weighted by their number of valid cells, and empty denominators left out of the sums.

The CLUE-S allocation of DisSLUCC was not modified to match lulcc.

**Behaviours visible only in the code.** The random forest models receive a numeric 0/1 response and therefore run in regression mode. The CLUE-S loop has a second stopping condition based on the mean signed difference between demand and allocation; because the rounded demand sums to the number of cells, that mean is zero, and only the largest difference stops the iteration.

**Results shown only as figures.** The AUCs are printed in the figures, but the agreement budgets and figures of merit are only drawn (Figs. 7–8). We compare the latter with our own runs of lulcc, not with the published curves.

## 5 Computational set-up

**Data.** The replication reads the Plum Island Ecosystems data set from GeoTIFF files exported from the R package (`data(pie)`), without any other change; the exported layers carry the original CRS (NAD83, Massachusetts Mainland, EPSG:26986).

**Reference.** We ran the demonstration script with lulcc 1.0.4[13] and archived the outputs[14]. Two references were produced:

- a *deterministic* one, with the randomness removed (seed 1 for the training sample, `jitter.f` = 0, ordered allocation not stochastic), exported stage by stage: training cells, regression coefficients, the suitability matrix, the demand table, all simulated maps and the three-map tables, with doubles written to 17 significant digits;
- an *ensemble* of 30 runs of the script as published, seeds 1 to 30.

The only change to the script is the repair described in Section 9.

**Software.** Reference: R 4.3.3, lulcc 1.0.4, raster 3.6-26, caret 6.0-94, ROCR 1.0-11, randomForest 4.7-1.1. Replication: Python 3.13, DisSModel 0.6.6, DisSLUCC 0.5.0, NumPy 2.5, SciPy 1.18, statsmodels 0.15, rasterio 1.5.

**Hardware and time.** Both were run on one Linux machine with two cores of an Intel Xeon at 2.1 GHz and 7 GB of memory, partly at the same time. One run of CLUE-S over the 14 steps took on average 12 s in lulcc (whose inner loop is in C) and 30 s in the replication; the ordered allocation took 3 s and 0.2 s. One complete run of the paper's experiment, including the regression trees and random forests of lulcc, took about 90 s in R; one run of the replication, about 30 s.

# 6 Implementation

Suitability is fitted with statsmodels (binomial GLM). The AUC is computed by the Mann–Whitney statistic, with ties counted as one half. CLUE-S is the allocation component of DisSLUCC (`AllocationDClueSLike`) on a DisSModel raster backend; the simulation starts at time 1, as lulcc does, and the suitability with elasticity is supplied to it in double precision. The random perturbation is added by a subclass of the DisSLUCC component that draws a new uniform number for every cell and class at every iteration. The ordered allocation and the three-map comparison are separate modules, operating on the valid cells in the raster's row-major order, which is also lulcc's cell order.

# 7 Comparison and acceptance criteria

Criteria were recorded in the code repository before either ensemble was run; the commit that adds them precedes those with results.

**Deterministic level.** Each stage is fed with the reference's own inputs, so that a difference points at that stage only. Coefficients and suitability must agree within numerical precision; demand, allocated maps and validation tables must be identical or agree to rounding error.

**Ensemble level.** Our 30 runs (seeds 1 to 30 of NumPy's default generator) are compared with lulcc's 30 on 20 metrics: the AUC of the glm for each class and for the gain of built, and the figure of merit of forest to built for each allocation method at each of the eight resolutions. A metric passes if the means differ by at most three standard errors of the difference and a two-sample Kolmogorov–Smirnov test gives $p \geq 0.01$. The regression trees and random forests of the paper's Fig. 5 are outside the criteria: we did not reimplement R's rpart and randomForest, and other implementations would differ by construction.

# 8 Results

## 8.1 Deterministic level

Table 1 lists the stages. The CLUE-S allocation of DisSLUCC, given lulcc's suitability and demand, produced the same map as lulcc at each of the 14 time steps, cell by cell; it needed 51 iterations at the first step and 257 at the last. The ordered allocation also matched at every step, and the three-map tables and figures of merit agreed to rounding error. Running the whole chain from our own regression gave the same final map: the coefficients differ by about $10^{-9}$ in relative terms, which did not change any allocation.

| Stage | Result |
|---|---|
| Observed transitions 1985–1999 (paper, Sect. 3.2) | identical |
| Regression coefficients, same training cells | max. relative difference $1.3 \times 10^{-9}$ |
| Suitability from lulcc's coefficients | max. difference $9 \times 10^{-14}$ |
| Suitability from our coefficients | max. difference $2.8 \times 10^{-10}$ |
| Demand, $t = 0, \ldots, 14$ | identical |
| CLUE-S, 14 steps | 0 cells differ at every step |
| Ordered allocation, 14 steps | 0 cells differ at every step |
| Three-map tables, 8 resolutions, both methods | max. difference $1.1 \times 10^{-16}$ |
| Figure of merit forest → built | max. difference $2.2 \times 10^{-16}$ |
| Whole chain from our own regression | final map identical |

**Table 1.** Deterministic level: each stage fed with the reference's inputs.

## 8.2 Ensemble level

All 20 metrics passed. Table 2 shows a selection and Figure 1 the figure of merit at every resolution; the spread between runs is the same in both implementations, including the larger spread of the stochastic ordered allocation.

| Metric | lulcc mean (sd) | replication mean (sd) | diff. | tol. | KS $p$ |
|---|---|---|---|---|---|
| AUC glm, forest | 0.6439 (0.0007) | 0.6437 (0.0006) | **−0.0003** | 0.0005 | 0.07 |
| AUC glm, built | 0.9382 (0.0003) | 0.9382 (0.0003) | **−0.0001** | 0.0002 | 0.81 |
| AUC glm, other | 0.7295 (0.0010) | 0.7290 (0.0008) | **−0.0004** | 0.0007 | 0.14 |
| AUC glm, gain of built | 0.6446 (0.0004) | 0.6446 (0.0004) | **0.0000** | 0.0003 | 0.81 |
| FoM CLUE-S, factor 2 | 0.1168 (0.0017) | 0.1167 (0.0019) | **−0.0001** | 0.0014 | 0.81 |
| FoM CLUE-S, factor 16 | 0.3766 (0.0058) | 0.3756 (0.0062) | **−0.0010** | 0.0046 | 0.81 |
| FoM CLUE-S, factor 256 | 0.7684 (0.0052) | 0.7688 (0.0052) | **+0.0004** | 0.0040 | 0.96 |
| FoM ordered, factor 2 | 0.1079 (0.0017) | 0.1079 (0.0019) | **0.0000** | 0.0014 | 1.00 |
| FoM ordered, factor 16 | 0.4311 (0.0188) | 0.4308 (0.0199) | **−0.0003** | 0.0150 | 0.96 |
| FoM ordered, factor 256 | 0.6795 (0.0326) | 0.6784 (0.0327) | **−0.0011** | 0.0253 | 1.00 |

**Table 2.** Ensemble level, 30 runs each (selection of the 20 metrics; all passed). FoM: figure of merit of the forest-to-built transition at 1999. Tolerance: three standard errors of the difference of means.

## 8.3 The published values

The values printed in the paper fall inside lulcc's own spread with one exception. In our 30 runs of lulcc, the paper's AUC for built (0.9384) is at the 70th percentile, that for the gain of built (0.6448) at the 63rd and that for other (0.7276) at the 7th; the AUC for forest (0.6423) is slightly below the lowest of the 30 runs (0.6425). The paper used lulcc 1.0 with the R of 2015; our reference used lulcc 1.0.4 on current R. We did not investigate the difference further.

# 9 Obstacles

**The demonstration script does not run unchanged.** With current versions of the raster and sp packages, the CLUE-S and ordered allocations stop with `different crs`. The rasters of the data set were serialised by an old version of raster, before the slot that holds the coordinate reference system in WKT form existed; the allocation creates each simulated map from the initial one, the new object carries a CRS in the current form and the old one does not, and the check that precedes stacking them fails. Reassigning the CRS of the old objects does not help: it leaves them with no CRS at all. Rebuilding each layer as a new raster object with the same grid, values and CRS solves it; this is the only

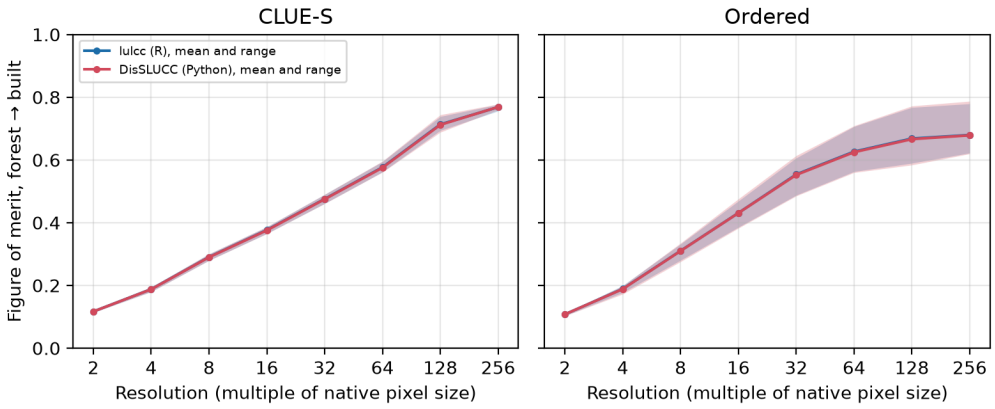



**Figure 1.** Figure of merit of the forest-to-built transition against resolution, as in Fig. 8 of the original article: mean and range of 30 runs of lulcc and of the replication.

change we made to the script. A package written to make land use models reproducible thus needed a repair to run its own published example eleven years later, for a reason unrelated to the model.

**An undeclared dependency.** lulcc loads the `gsubfn` package when it builds the explanatory variables, but lists it only as a suggestion; it has to be installed by hand.

## 10 Conclusion

The Plum Island Ecosystems application of lulcc was replicated. With the same inputs, an independent implementation of CLUE-S reproduces lulcc's maps exactly, and so do our ordered allocation and validation; with independent random numbers, the two implementations agree in distribution on every metric compared. The main obstacle was not in the model but in the environment around it.

## Reproducing this replication

Code: https://github.com/LambdaGeo/rescience-lulcc. Reference outputs and the Docker environment that produces them: https://github.com/LambdaGeo/lulcc-goldens. With the reference downloaded, `make env`, `make check` (about one minute), `make ensemble` (about 15 minutes) and `make compare` reproduce Tables 1 and 2 and Figure 1. The software versions are fixed in the repository.

## Acknowledgements

The code and this text were developed with the assistance of an AI model (Claude, Anthropic); all results were checked by running the code.
*Complete table of the 20 ensemble metrics: see* `results/ensemble_comparison.csv` *in the code repository.*

# References


1. S. Moulds, W. Buytaert, and A. Mijic. "An open and extensible framework for spatially explicit land use change modelling: the lulcc R package." In: **Geoscientific Model Development** 8.10 (2015), pp. 3215–3229. DOI: 10.5194/gmd-8-3215-2015.
2. P. H. Verburg, W. Soepboer, A. Veldkamp, R. Limpiada, V. Espaldon, and S. S. A. Mastura. "Modeling the spatial dynamics of regional land use: the CLUE-S model." In: **Environmental Management** 30.3 (2002), pp. 391–405. DOI: 10.1007/s00267-002-2630-x.
3. R. Fuchs, M. Herold, P. H. Verburg, and J. G. P. W. Clevers. "A high-resolution and harmonized model approach for reconstructing and analysing historic land changes in Europe." In: **Biogeosciences** 10 (2013), pp. 1543–1559. DOI: 10.5194/bg-10-1543-2013.
4. R. G. Pontius, S. Peethambaram, and J.-C. Castella. "Comparison of three maps at multiple resolutions: a case study of land change simulation in Cho Don District, Vietnam." In: **Annals of the Association of American Geographers** 101.1 (2011), pp. 45–62. DOI: 10.1080/00045608.2010.517742.
5. National Academies of Sciences, Engineering, and Medicine. **Reproducibility and Replicability in Science**. The National Academies Press, 2019. DOI: 10.17226/25303.
6. F.-R. Mazy and P.-Y. Longaretti. "Towards a generic theoretical framework for pattern-based LUCC modeling: allocation revisited: formal foundations and bias identification." In: **Environmental Modelling & Software** 166 (2023), p. 105706. DOI: 10.1016/j.envsoft.2023.105706.
7. S. S. Costa, N. J. Santos Junior, F. M. Sousa, and D. S. Bezerra. **DisSModel: A Python framework for spatially explicit dynamic modeling**. Version 0.6.6. 2026. DOI: 10.5281/zenodo.23246106.
8. S. S. Costa. **DisSLUCC**. Version 0.5.0. 2026. DOI: 10.5281/zenodo.23219338.
9. A. P. D. Aguiar, T. Carneiro, P. Andrade, and T. Assis. "LuccME-TerraME: an open-source framework for spatially explicit land use change modelling." In: **GLP News** 8 (2012), pp. 21–23.
10. T. G. d. S. Carneiro, P. R. d. Andrade, G. Câmara, A. M. V. Monteiro, and R. R. Pereira. "An extensible toolbox for modeling nature–society interactions." In: **Environmental Modelling & Software** 46 (2013), pp. 104–117. DOI: 10.1016/j.envsoft.2013.03.002.
11. S. S. Costa. **DisSLUCC benchmark**. Version 0.2.2. 2026. DOI: 10.5281/zenodo.23224097.
12. R. G. Pontius and B. Parmentier. "Recommendations for using the relative operating characteristic (ROC)." In: **Landscape Ecology** 29.3 (2014), pp. 367–382. DOI: 10.1007/s10980-013-9984-8.
13. S. Moulds. **lulcc: Land Use Change Modelling in R**. R package version 1.0.4. 2019. URL: https://cran.r-project.org/package=lulcc.
14. S. S. Costa. **lulcc-goldens: reference outputs of lulcc 1.0.4 for the Plum Island Ecosystems application**. 2026. URL: https://github.com/LambdaGeo/lulcc-goldens.